

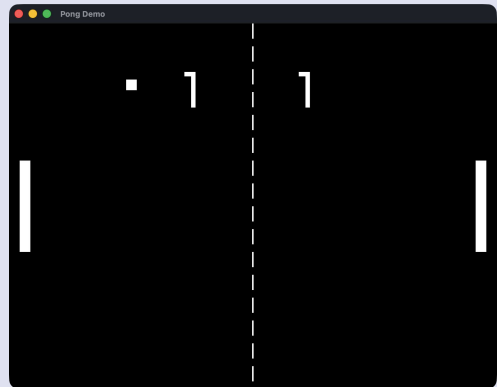
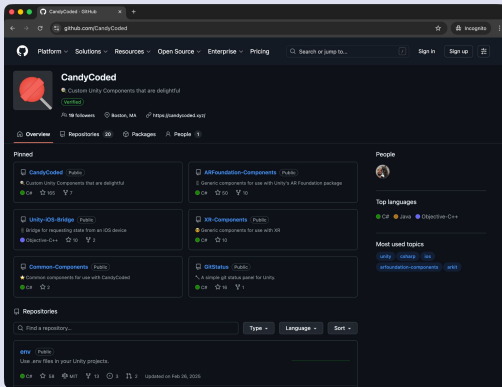
How to Build a Rhythm Game in (Almost) Any Game Engine

Boston Game Dev Week – MiniCon 2026

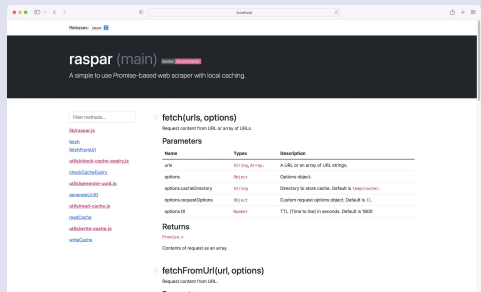
Who am I?

- Web / Game Developer
- Worked with Unity, Unreal, and Godot
- Built games as well as consumer based products in game engines
- Release a lot of what I do as open source projects
- I love data-in-data-out systems, writing unit tests and regular expressions (seriously)

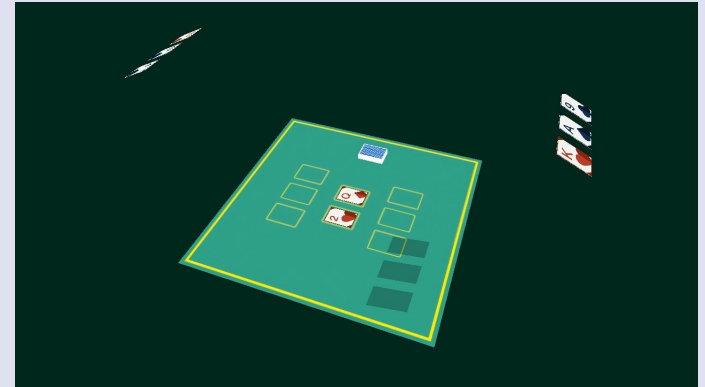
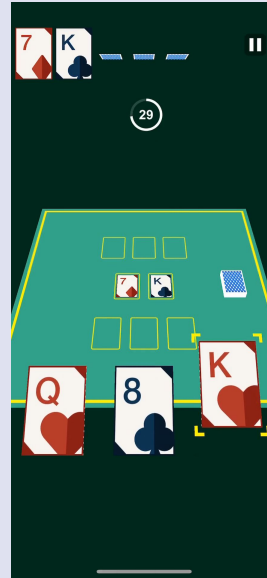
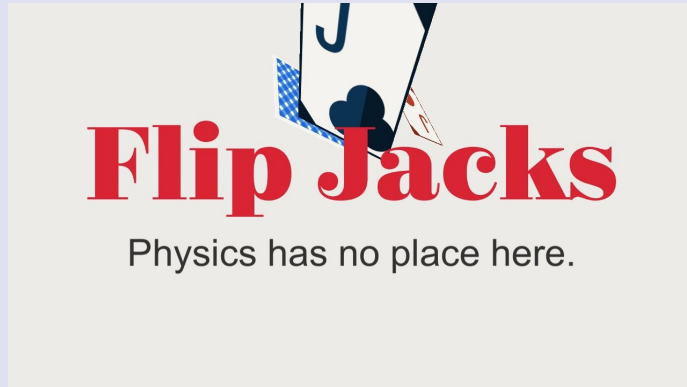




And many, many more tools.



What Have I Built (Games)



<https://flipjacksgame.com/>

My Favorite Rhythm Games

- PaRappa the Rapper (Playstation 1997)
- Guitar Hero (PS2 - 2005)
- Rock Band (XBOX - 2007)
- Patapon (PSP - 2008)
- DJ Hero (XBOX - 2009)
- Amplitude (PS4 - 2016)
- Audica (VR - 2019)
- Hi-Fi Rush (PC - 2023)

My Favorite Rhythm Games

- PaRappa the Rapper (Playstation 1997)
- Guitar Hero (PS2 - 2005)
- Rock Band (XBOX - 2007)
- Patapon (PSP - 2008)
- DJ Hero (XBOX - 2009)
- Amplitude (PS4 - 2016)
- Audica (VR - 2019)
- Hi-Fi Rush (PC - 2023)

S	       
A	
B	
C	
D	

Disclaimer:
I'm awful at rhythm games

Disclaimer:

**I'm awful at rhythm games
& I'm not a musician.** 🤪

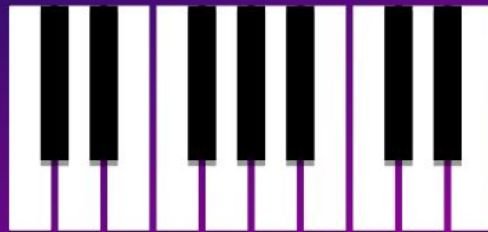
But I've talked to people who are:

1. Good at rhythm games

2. Actual musicians

Rhythm Game Utilities

A collection of utilities for creating rhythm games in Unity, Unreal, Godot, SDL and MonoGame.



<https://rhythmgameutilities.com/>

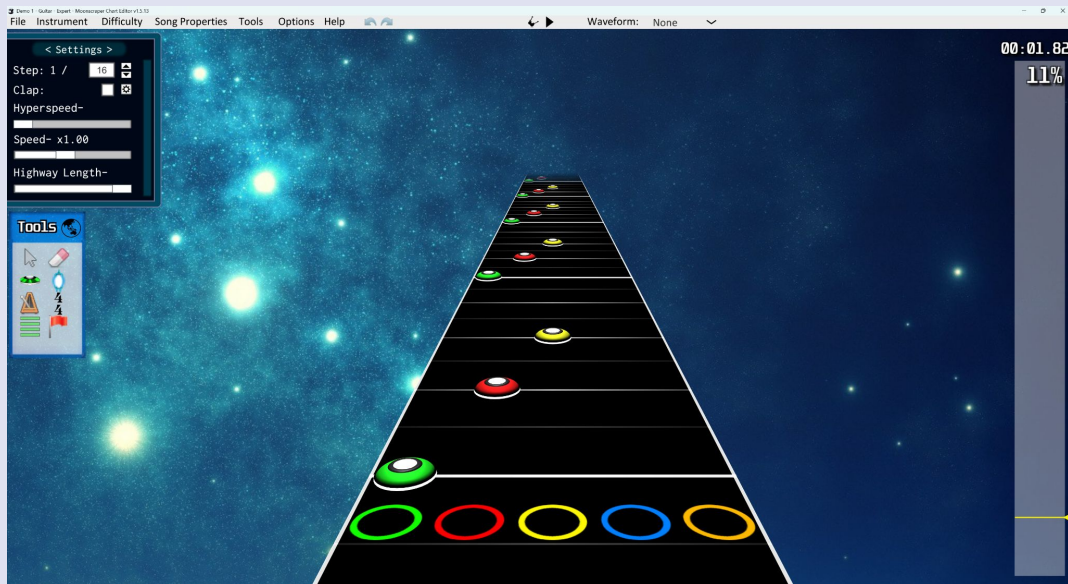
Let's Break Down Rhythm Games

Rhythm Games Have Complicated Systems

1. Charting Notes
2. Loading Notes
3. Rendering Notes
4. Syncing Notes With Audio
5. Calculating Accuracy From User Input
6. Beat Matching
7. (Bonus) Input Devices (Midi/Custom)
8. (Bonus) Handling Input Latency

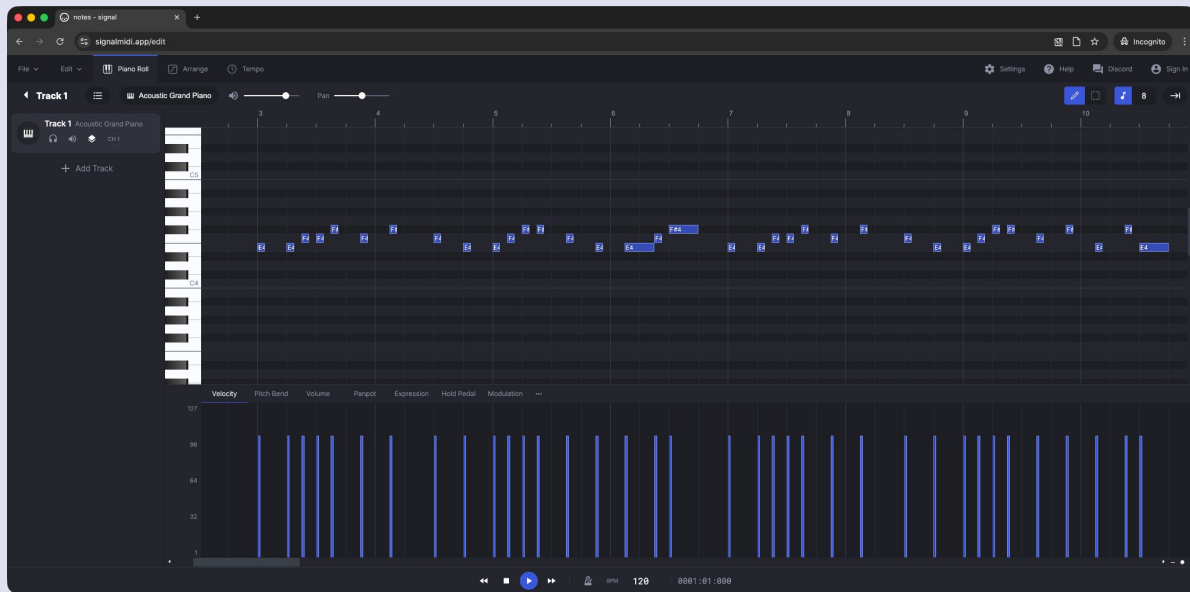
1. Charting Notes

1. Charting Notes (.chart)



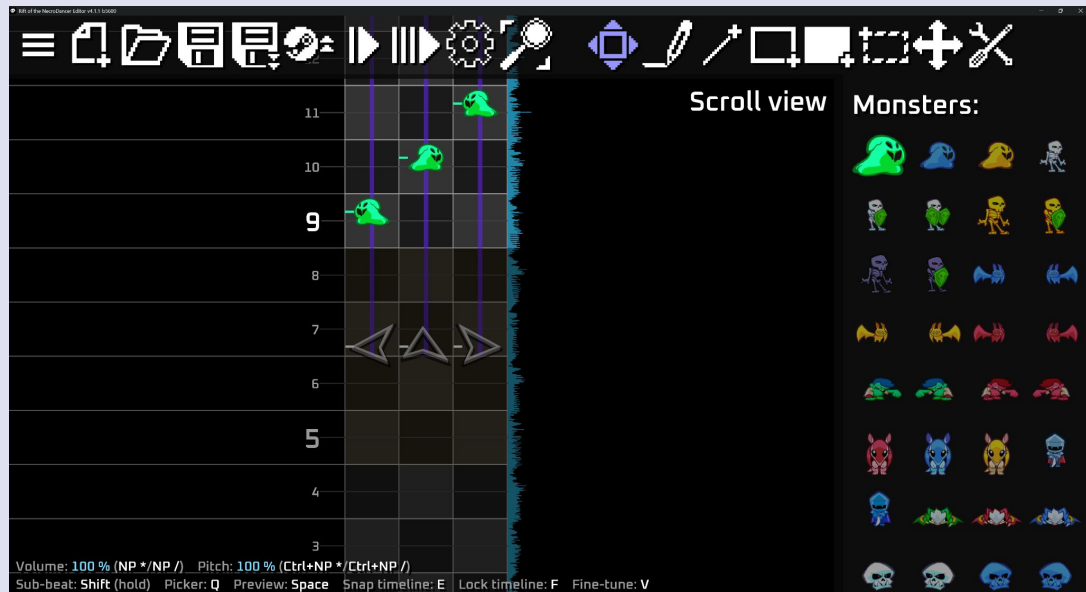
<https://github.com/FireFox2000000/Moonscraper-Chart-Editor>

1. Charting Notes (.mid)



<https://signalmidi.app/>

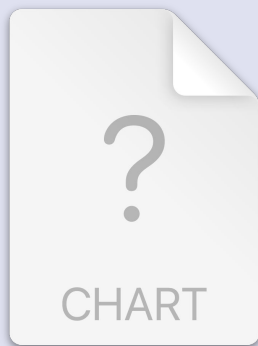
1. Charting Notes (In Game)



Rift of the NecroDancer Level Editor

2. Loading Notes

2. Loading Notes (.chart)



song.chart



```
[Song]
{
  Name = "Demo 1"
  Artist = "Scott Doxey"
  Charter = "Scott Doxey"
  Album = "Demo"
  Year = ", 2024"
  Offset = 0
  Resolution = 192
  Player2 = bass
  Difficulty = 0
  PreviewStart = 0
  PreviewEnd = 0
  Genre = "rock"
  MediaType = "cd"
  MusicStream = "song.ogg"
}

[SyncTrack]
{
  0 = TS 4
  0 = B 120000
}

[ExpertSingle]
{
  768 = N 0 0
  960 = N 1 0
  1152 = N 2 0
  1536 = N 0 0
  1728 = N 1 0
  1920 = N 2 0
  2304 = N 0 0
  2496 = N 1 0
  2688 = N 2 0
  3072 = N 0 0
  3264 = N 1 0
  3456 = N 2 0
  3840 = N 0 0
  4032 = N 1 0
  4224 = N 2 0
  4608 = N 0 0
  4800 = N 1 0
  4992 = N 2 0
  5376 = N 0 0
  5568 = N 1 0
  5760 = N 2 0
}
```

2. Loading Notes (.chart)

```
[SyncTrack]
{
  0 = TS 4
  0 = B 120000
}
```

```
[ExpertSingle]
{
  768 = N 0 0
  960 = N 1 0
  1152 = N 2 0
  1536 = N 0 0
  ...
}
```

First value is the
tick (position)

2. Loading Notes (.chart)

```
[SyncTrack]
{
  0 = TS 4           TS = Time Signature
  0 = B 120000       B = BPM
}
```

2. Loading Notes (.chart)

```
[ExpertSingle]
```

```
{
```

```
768 = N 0 0
```

```
960 = N 1 0
```

```
1152 = N 2 0
```

```
1536 = N 0 0
```

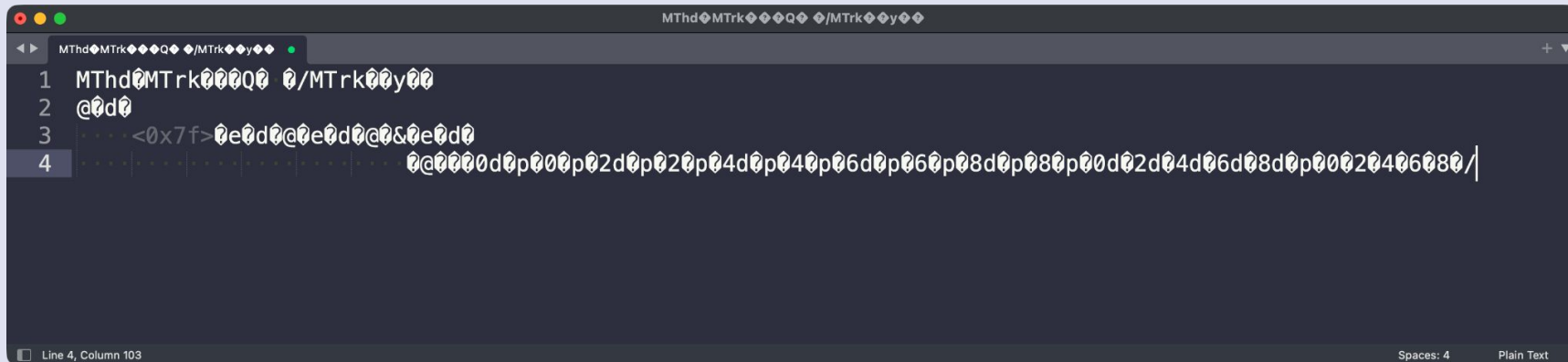
```
...
```

```
}
```

Note
Hand Position
Length (Hold Notes)

N 1 0

2. Loading Notes (.mid)

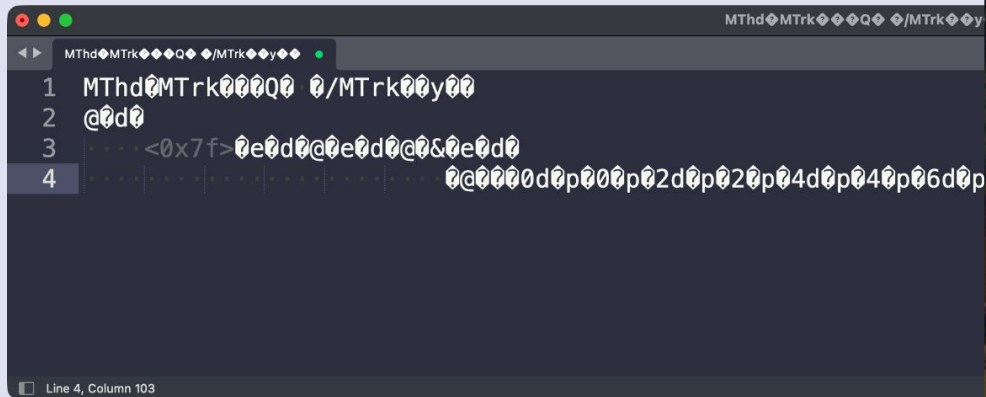


A screenshot of a code editor window with a dark theme. The title bar shows the file path: `MThdMTrkQy/MTrkyy`. The editor contains four lines of text:

```
1 MThdMTrkQy /MTrkyy
2 @d
3 <0x7f>ed@ed@e
4 @000d0p00p02d0p020p04d0p04p06d0p06p08d0p08p00d02d04d06d08d0p0002040608/
```

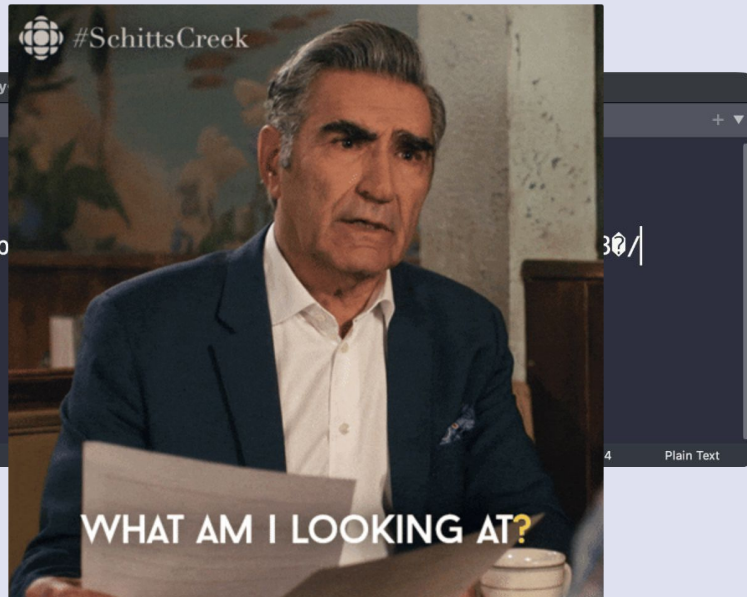
The fourth line is highlighted. The status bar at the bottom indicates "Line 4, Column 103", "Spaces: 4", and "Plain Text".

2. Loading Notes (.mid)



A screenshot of a code editor window with a dark theme. The title bar shows a file named 'MThdMTrk...'. The editor contains four lines of MIDI data: Line 1: 'MThdMTrk00000 0/MTrk00y00', Line 2: '@0d0', Line 3: '<0x7f>0e0d0@e0d0@0e0d0', and Line 4: '@0000d0p000p02d0p020p04d0p040p06d0p'. The fourth line is highlighted. The status bar at the bottom indicates 'Line 4, Column 103'.

```
1 MThdMTrk00000 0/MTrk00y00
2 @0d0
3 <0x7f>0e0d0@e0d0@0e0d0
4 @0000d0p000p02d0p020p04d0p040p06d0p
```



2. Loading Notes (.chart)



song.chart



```
[Song]
{
  Name = "Demo 1"
  Artist = "Scott Doxey"
  Charter = "Scott Doxey"
  Album = "Demo"
  Year = ", 2024"
  Offset = 0
  Resolution = 192
  Player2 = bass
  Difficulty = 0
  PreviewStart = 0
  PreviewEnd = 0
  Genre = "rock"
  MediaType = "cd"
  MusicStream = "song.ogg"
}

[SyncTrack]
{
  0 = TS 4
  0 = B 120000
}

[ExpertSingle]
{
  768 = N 0 0
  960 = N 1 0
  1152 = N 2 0
  1536 = N 0 0
  1728 = N 1 0
  1920 = N 2 0
  2304 = N 0 0
  2496 = N 1 0
  2688 = N 2 0
  3072 = N 0 0
  3264 = N 1 0
  3456 = N 2 0
  3840 = N 0 0
  4032 = N 1 0
  4224 = N 2 0
  4608 = N 0 0
  4800 = N 1 0
  4992 = N 2 0
  5376 = N 0 0
  5568 = N 1 0
  5760 = N 2 0
}
```


2. Loading Notes (.chart)

```
1 inline std::regex CHART_SECTION_PATTERN(R"(\[[a-z]+\]\s*\{([^\}]+\)\})",
2                                         std::regex_constants::icase);
3
4 inline std::regex CHART_SECTION_LINE_PATTERN(R"((^[=]+)\s*=(^[r\n]+))");
5
6 inline std::regex JSON_VALUE_PATTERN(R"((("[^"]+"|\s+))");
```

Yes, I actually wrote these by hand because I'm nuts.

2. Loading Notes

- `Song.FromChartData`
- `Song.FromMidiData`
- `ReadNotesFromChartData`
- `ReadNotesFromMidiData`

```
npx chart-to-json ./notes.chart > notes.json
```

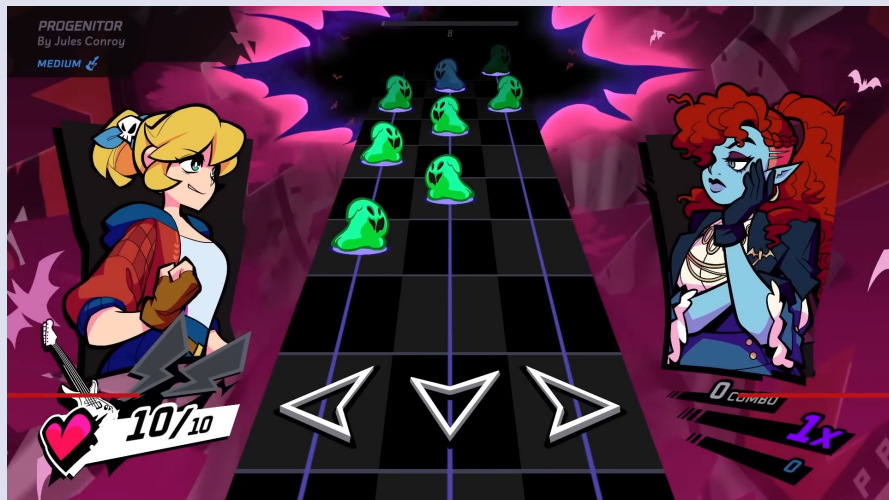
```
{
  "Name": "Demo 1",
  "Artist": "Scott Doxey",
  "Album": "Demo",
  "Genre": "rock",
  "Year": ", 2024",
  "Charter": "Scott Doxey",
  "Resolution": 192,
  "Difficulty": 0,
  "Offset": 0,
  "PreviewStart": 0,
  "PreviewEnd": 0,
  "MusicStream": "song.ogg",
  "Lyrics": {},
  ...
}

{
  ...,
  "BPM": {
    "0": 120000
  },
  "TimeSignatures": {
    "0": [
      4
    ]
  }
}

{
  ...,
  "Difficulties": {
    ...,
    "Expert": [
      {
        "Position": 768,
        "HandPosition": 0,
        "Length": 0
      },
      {
        "Position": 960,
        "HandPosition": 1,
        "Length": 0
      },
      {
        "Position": 1152,
        "HandPosition": 2,
        "Length": 0
      },
      {
        "Position": 1536,
        "HandPosition": 0,
        "Length": 0
      },
      ...
    ]
  },
  ...
}
```

3. Rendering Notes

3. Rendering Notes



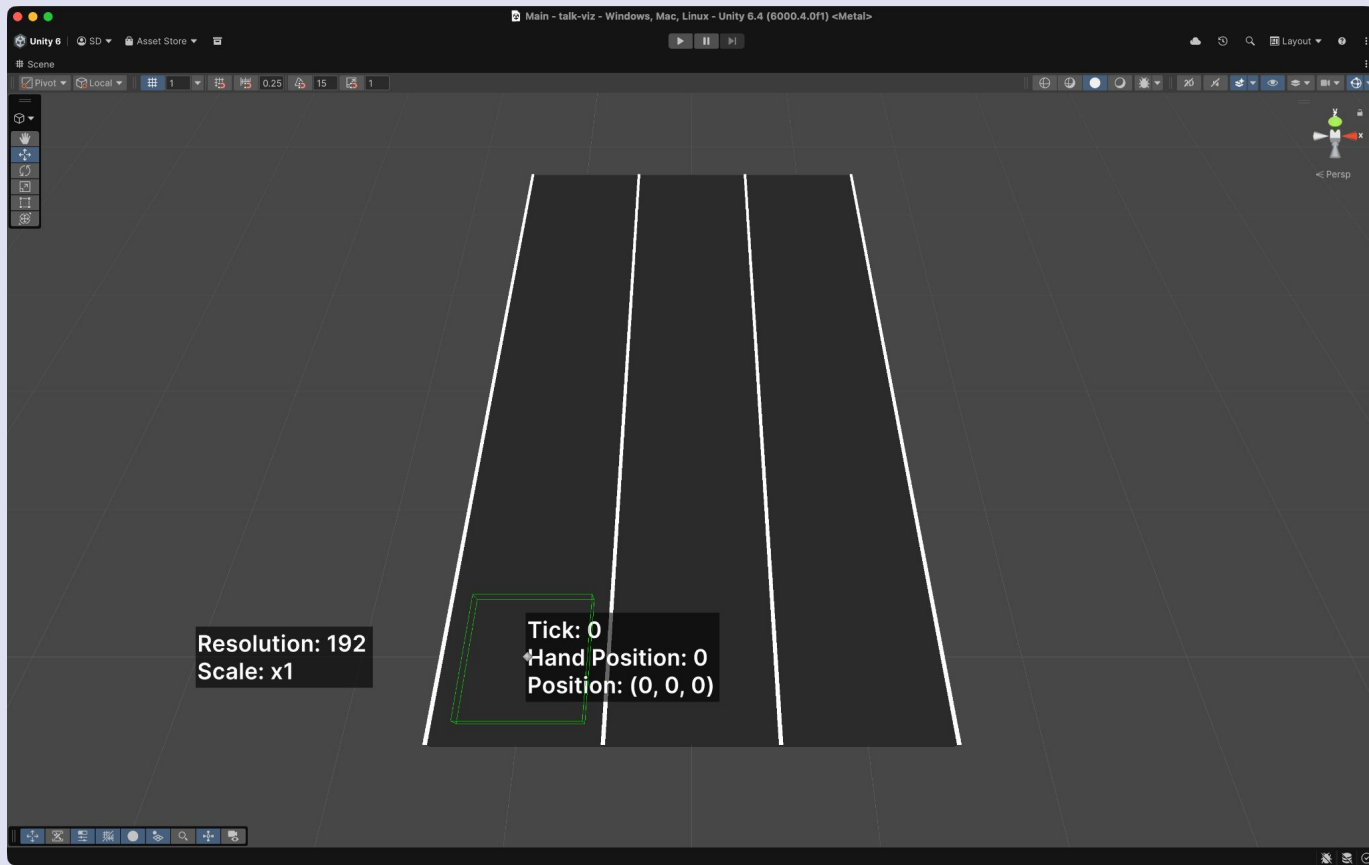
Rift of the NecroDancer



Amplitude (PS4)

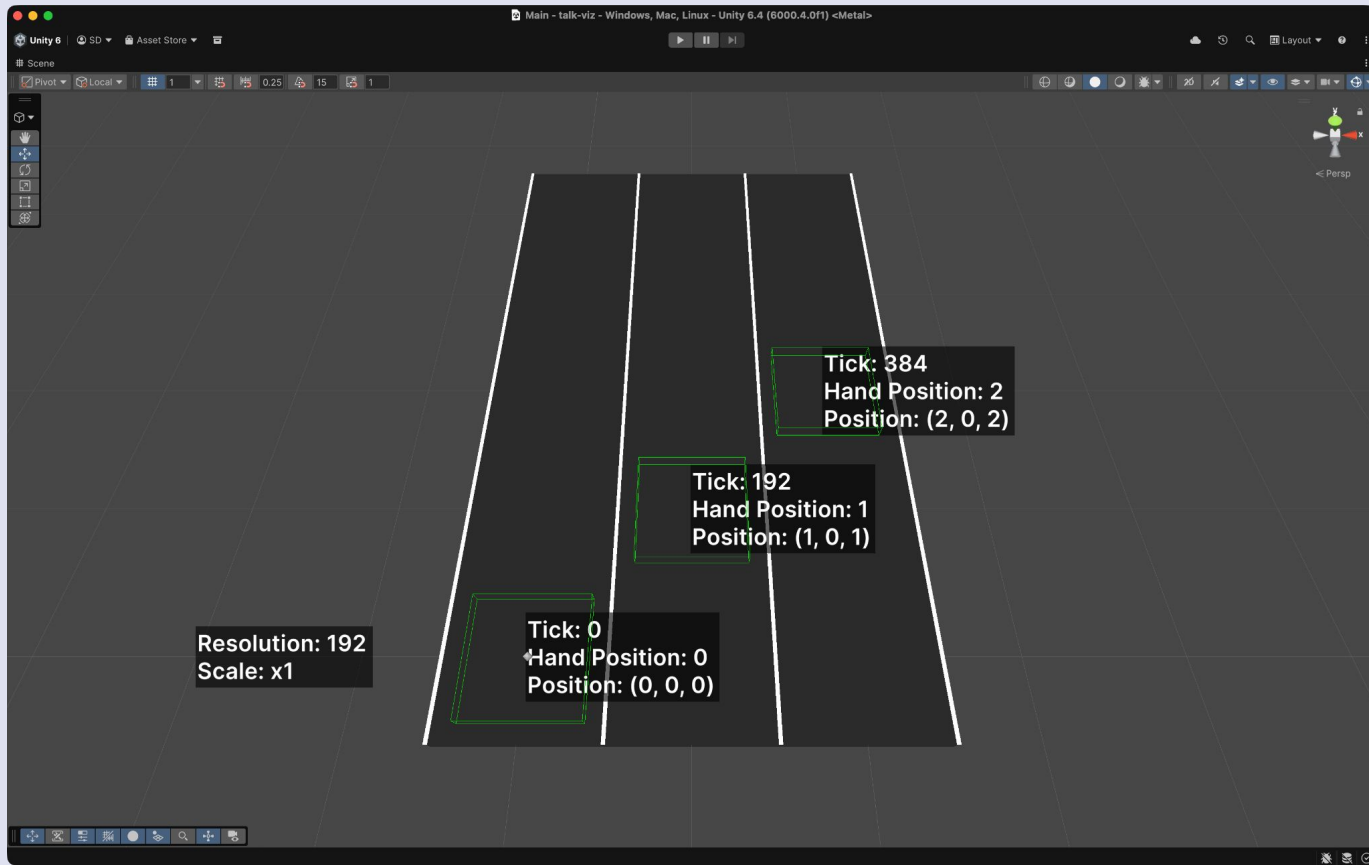
3. Rendering Notes

```
1 using System;
2 using RhythmGameUtilities;
3
4 const int tick = 0;
5 const int resolution = 192;
6
7 var position = Utilities.ConvertTickToPosition(tick, resolution);
8
9 Console.WriteLine(position); // 0
```



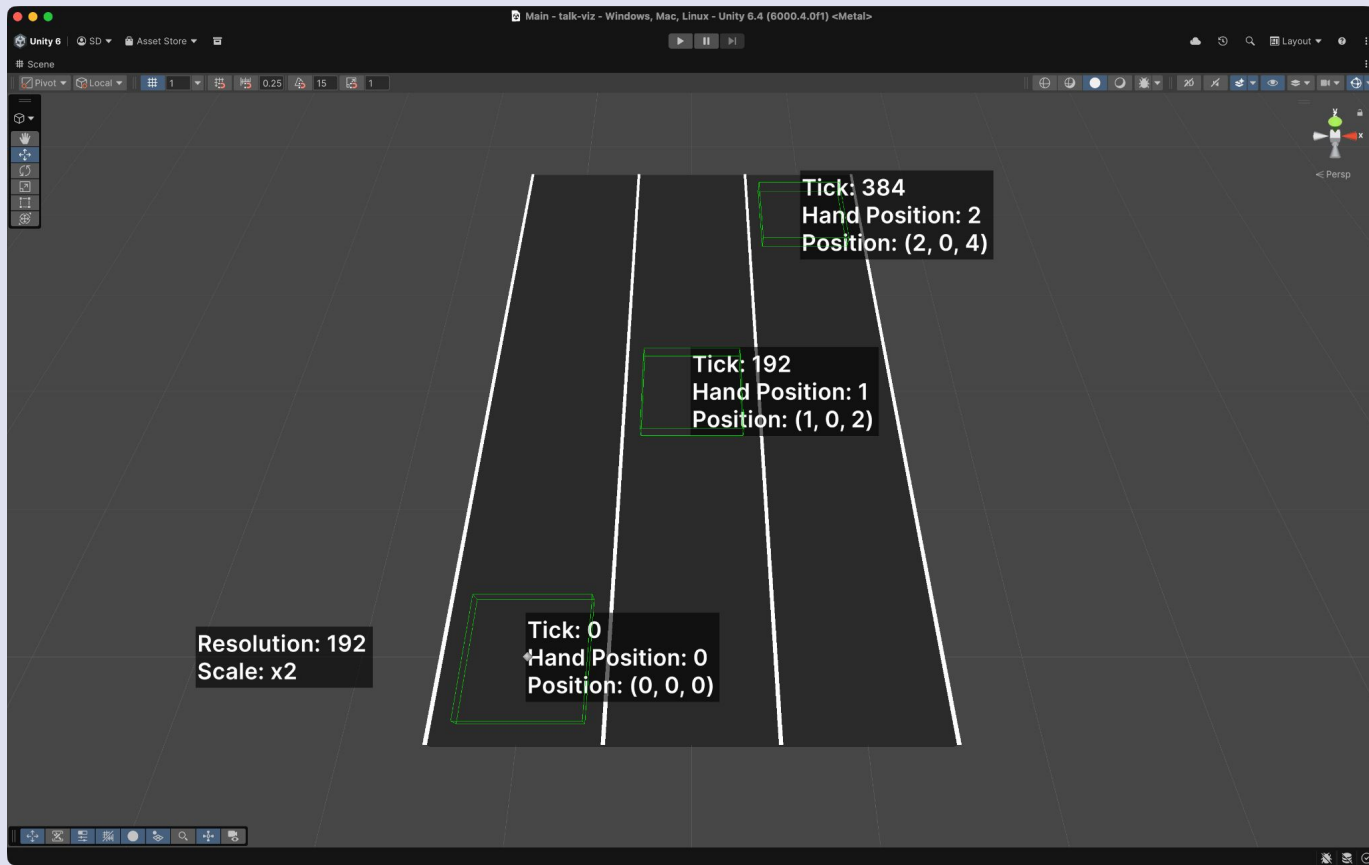
3. Rendering Notes

```
1 using System;
2 using RhythmGameUtilities;
3
4 const int resolution = 192;
5
6 Console.WriteLine(Utilities.ConvertTickToPosition(192, resolution)); // 1
7 Console.WriteLine(Utilities.ConvertTickToPosition(384, resolution)); // 2
```



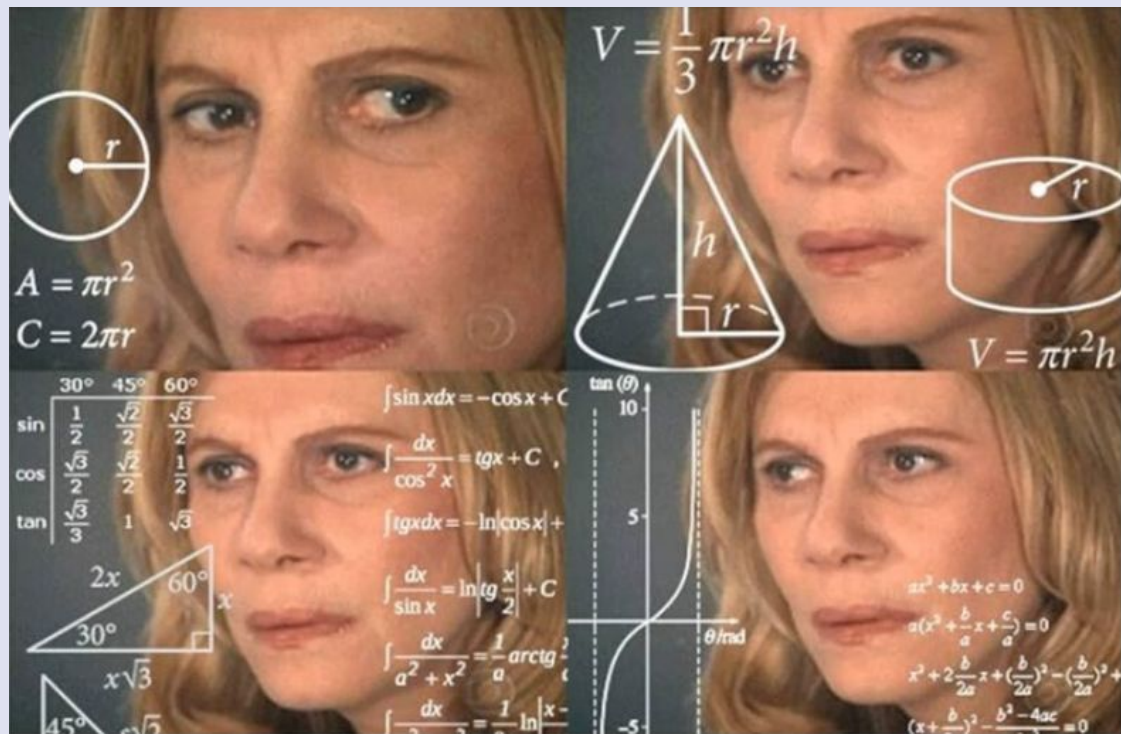
3. Rendering Notes

```
1 using System;
2 using RhythmGameUtilities;
3
4 const int tick = 384;
5 const int resolution = 192;
6 const int scale = 2
7
8 var position = Utilities.ConvertTickToPosition(tick, resolution) * scale;
9
10 Console.WriteLine(position); // 4
```



4. Syncing Notes With Audio

So this section gets a bit ...



Disclaimer:

**I'm awful at rhythm games
& I'm not a musician.** 🤪

Disclaimer:

**I'm awful at rhythm games
& I'm not a musician.** 🤪

(and I'm not great at math)

But I've talked to people who are:

1. Good at rhythm games

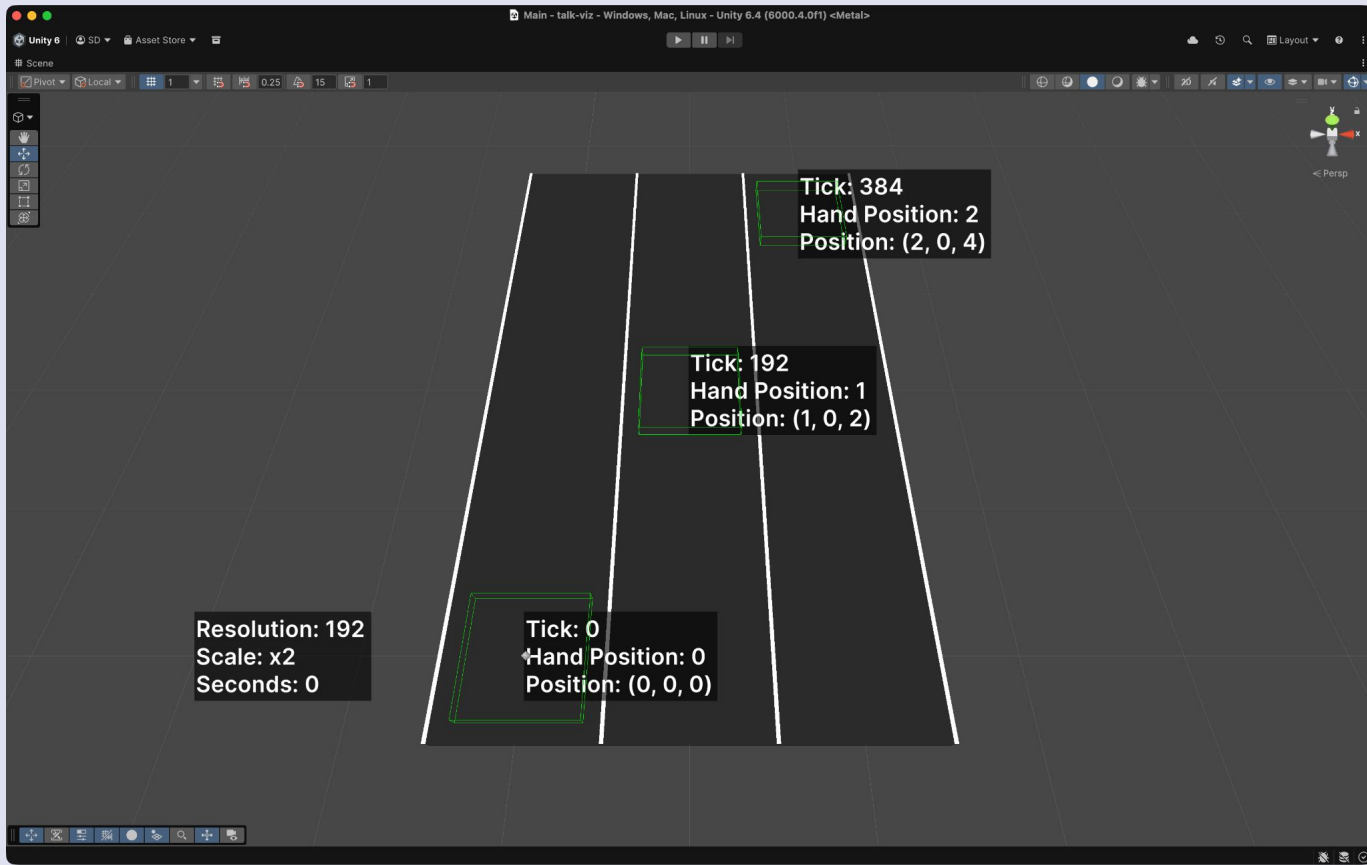
2. Actual musicians

But I've talked to people who are:

- 1. Good at rhythm games**
- 2. Actual musicians**
- 3. Good at math**

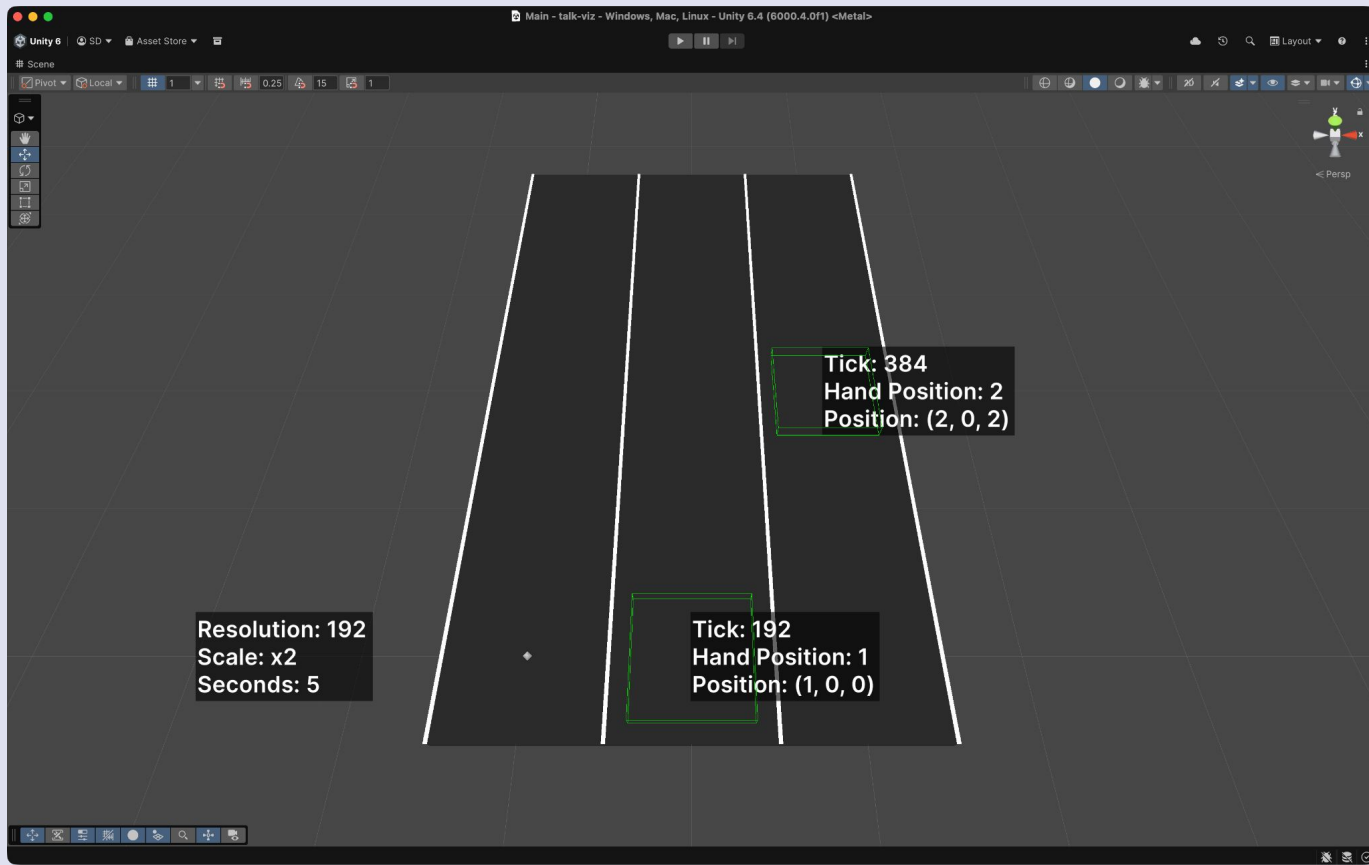
4. Syncing Notes With Audio

```
1 var currentTime = 0;
2
3 var resolution = 192;
4 var tempoChanges = new Tempo[] { new() { Position = 0, BPM = 12000 } };
5
6 var tickOffset = Utilities.ConvertSecondsToTicks(currentTime, resolution, tempoChanges);
7
8 Console.WriteLine(tickOffset); // 0
9
10 var note = new Note { Position = 0, HandPosition = 0, Length = 0 };
11
12 var position = Utilities.ConvertTickToPosition(note.Position - tickOffset, resolution);
13
14 Console.WriteLine(position); // 0
```



4. Syncing Notes With Audio

```
1 var currentTime = 5;
2
3 var resolution = 192;
4 var tempoChanges = new Tempo[] { new() { Position = 0, BPM = 12000 } };
5
6 var tickOffset = Utilities.ConvertSecondsToTicks(currentTime, resolution, tempoChanges);
7
8 Console.WriteLine(tickOffset); // 192
9
10 var note = new Note { Position = 0, HandPosition = 0, Length = 0 };
11
12 var position = Utilities.ConvertTickToPosition(note.Position - tickOffset, resolution);
13
14 Console.WriteLine(position); // -1
```





4. Syncing Notes With Audio

```
1 var currentTime = 5;
2
3 var resolution = 192;
4 var tempoChanges = new Tempo[]
5 {
6     new() { Position = 0, BPM = 88000 }, new() { Position = 3840, BPM = 112000 },
7     new() { Position = 9984, BPM = 89600 }, new() { Position = 22272, BPM = 112000 },
8     new() { Position = 33792, BPM = 111500 }, new() { Position = 34560, BPM = 112000 },
9     new() { Position = 42240, BPM = 111980 }
10 };
11
12 var tickOffset = Utilities.ConvertSecondsToTicks(currentTime, resolution, tempoChanges);
13
14 Console.WriteLine(tickOffset); // 1408
15
16 var note = new Note { Position = 0, HandPosition = 0, Length = 0 };
17
18 var position = Utilities.ConvertTickToPosition(note.Position - tickOffset, resolution);
19
20 Console.WriteLine(position); // -7.3333335
```



5. Calculating Accuracy From User Input

5. Calculating Accuracy of Hit Notes From User Input

```
1 var currentPosition = Utilities.ConvertSecondsToTicks(  
2     currentTime, resolution, tempoChanges);  
3  
4 var accuracyRatio = Utilities.CalculateAccuracyRatio(  
5     note.Position, currentPosition, delta);  
6  
7 Console.WriteLine(accuracy); // 0.64
```

5. Calculating Accuracy of Hit Notes From User Input



5. Calculating Accuracy of Hit Notes From User Input



5. Calculating Accuracy of Hit Notes From User Input

```
1 var currentPosition = Utilities.ConvertSecondsToTicks(  
2     currentTime, resolution, tempoChanges);  
3  
4 var accuracy = Utilities.CalculateAccuracy(note.Position,  
5     currentPosition, delta);  
6  
7 Console.WriteLine(accuracy); // Invalid, Poor, Fair, Good, Great, Perfect
```

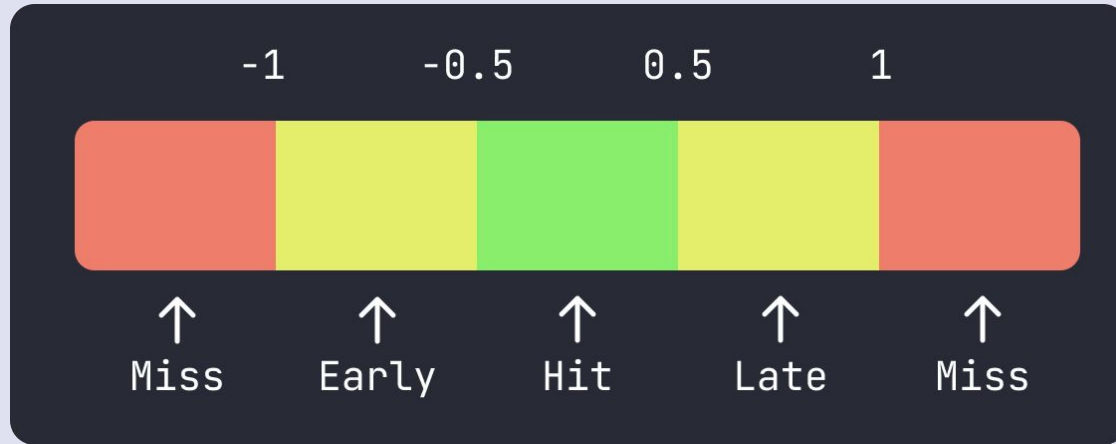
5. Calculating Accuracy of Hit Notes From User Input



5. Calculating Accuracy of Hit Notes From User Input

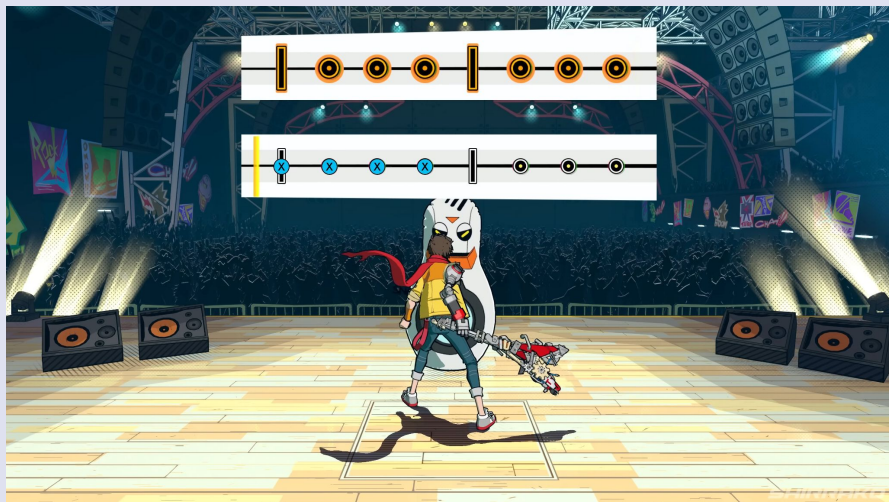
```
1 var currentPosition = Utilities.ConvertSecondsToTicks(  
2     currentTime, resolution, tempoChanges);  
3  
4 var timing = Utilities.CalculateTiming(  
5     note.Position, currentPosition, delta);  
6  
7 Console.WriteLine(timing); // Miss, Hit, Early, Late
```

5. Calculating Accuracy of Hit Notes From User Input

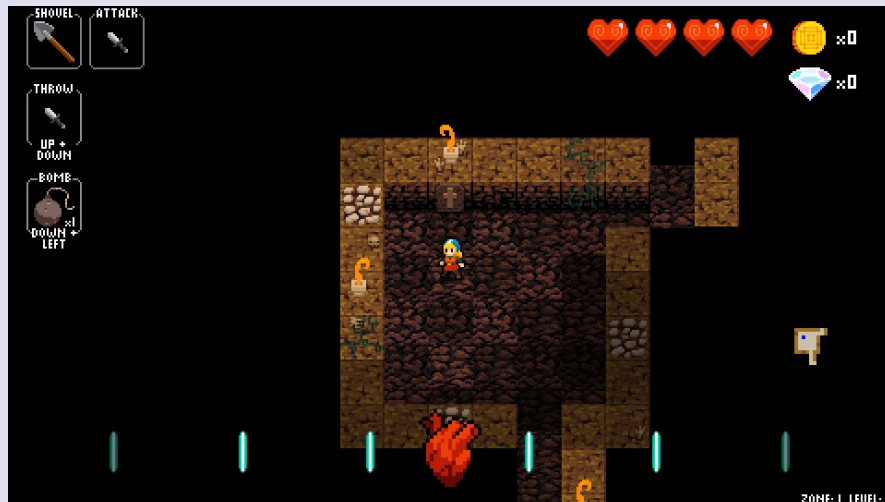


6. Beat Matching

6. Beat Matching



Hi-Fi Rush



Crypt of the NecroDancer

6. Beat Matching

```
1 using System;
2 using RhythmGameUtilities;
3
4 const int bpm = 120;
5 const float currentTime = 10f;
6 const float delta = 0.05f;
7
8 var isOnTheBeat = Utilities.IsOnTheBeat(bpm, currentTime, delta);
9
10 Console.WriteLine(isOnTheBeat); // true
```

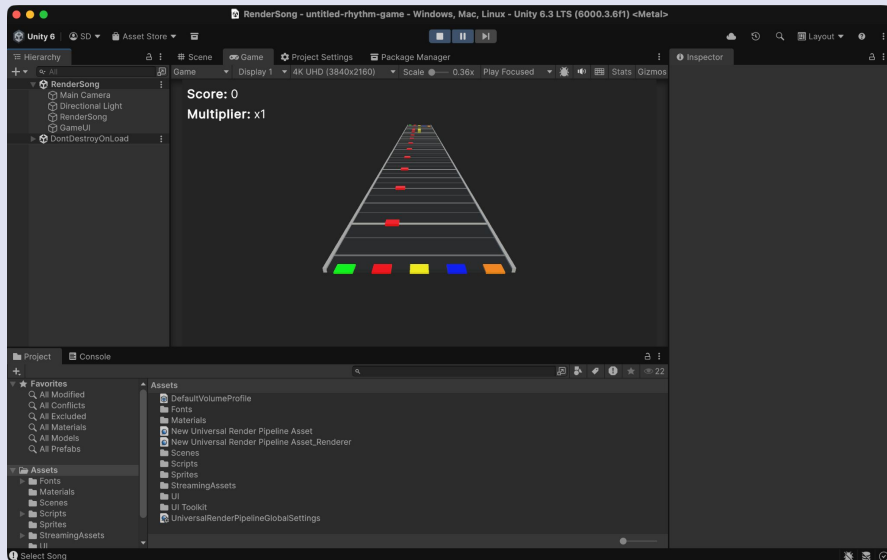
6. Beat Matching

```
1 private void Update()
2 {
3     elapsedTime += Time.deltaTime;
4
5     var isOnTheBeat = Utilities.IsOnTheBeat(120, elapsedTime);
6
7     if (!isOnBeatTriggered && isOnTheBeat)
8     {
9         Debug.Log("On Beat");
10
11         isOnBeatTriggered = true;
12     }
13     else if (isOnBeatTriggered && !isOnTheBeat)
14     {
15         isOnBeatTriggered = false;
16     }
17 }
```

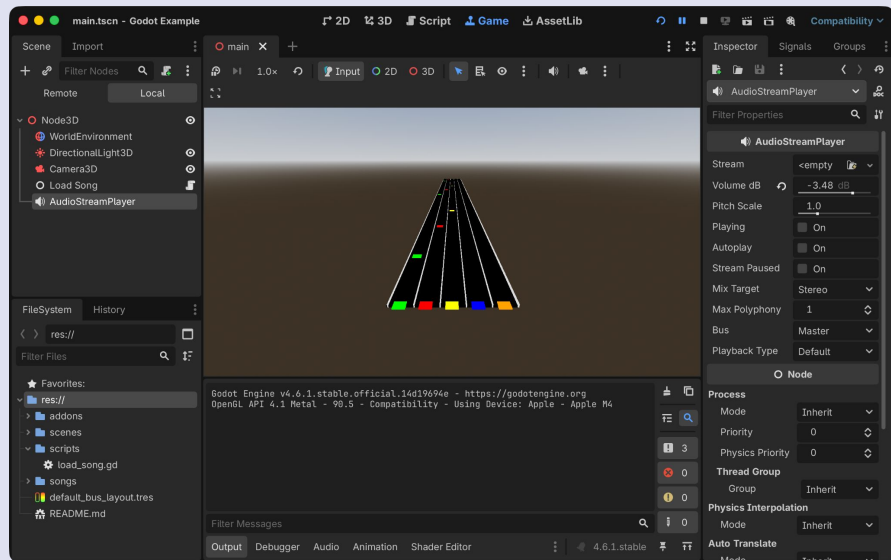
Upcoming Features

- Stepmania Import
- Love2D (Lua) Plugin
- Bevy (Rust) Plugin
- GameMaker (GML) Plugin
- Desktop/Web Charting Tool

Examples



Unity



Godot

Examples



SDL

Questions?

Thank you!

